Лабораторная работа «Проектирование веб-API»

Лабораторный практикум по ТРПО

1. Проектирование API для пользователей
2. Проектирование программного интерфейса
3. Описание API с помощью формата описания

ЧувГУ, каф. Вычислительной техники
Доц. , к.т.н. Ржавин В.В.

На сегодняшний день всё больше и больше IT-компаний понимают и принимают важность концепции «API-first», т.е. парадигмы разработки ПО, в которой главной составляющей является разработка API. Под этим мы будем понимать то, что

- *контракт разрабатывается и фиксируется в виде спецификации до того, как функциональность непосредственно реализована;*
- *если обнаруживается несоответствие контракта и его реализации, изменения вносятся в реализацию, а не в контракт.*

В данной лабораторной работе рассматривается проектирование только веб-API, как наиболее важной и распространенной части общего подхода к проектированию API.



ЧТО НАДО ЗНАТЬ

API (***a**pplication **p**rogramming **i**nterface* - дословно *интерфейс программирования приложения*) — программный интерфейс, то есть описание способов взаимодействия одной компьютерной программы с другими. Обычно входит в описание какого-либо интернет-протокола, программного каркаса (фреймворка) или стандарта вызовов функций операционной системы. Часто реализуется отдельной программной библиотекой или сервисом операционной системы. Используется программистами при написании всевозможных приложений.

Веб-API — это интерфейс прикладного программирования (API) либо для веб-сервера, либо для веб-браузера. Как концепция веб-разработки, она может быть связана с клиентской частью веб-приложения (включая любые используемые веб-фреймворки). Серверный веб-API состоит из одной или нескольких общедоступных **конечных точек** для определенной системы сообщений запрос–ответ, обычно выражаемой в JSON или XML посредством веб-сервера на основе HTTP. Серверный API (SAPI) не считается серверным веб-API, если только он не является общедоступным с помощью удаленного веб-приложения.

Википедия

Веб-API – это удаленные API, которые можно использовать с протоколом HTTP!

Работа API

API служат в качестве посредников, обеспечивающих бесперебойную связь и обмен данными между программными приложениями. Процесс начинается с того, что клиент, например, мобильное приложение или веб-приложение, инициирует запрос к API. Этот запрос содержит важную информацию, такую как конечная точка API, метод HTTP, а также, если требуется, учетные данные аутентификации и полезную нагрузку данных.

Получив запрос, сервер API обрабатывает его на основе заранее определенных правил. Это может включать маршрутизацию запроса в соответствующую службу, проверку данных и применение соответствующей бизнес-логики. Затем сервер API связывается с необходимыми внутренними службами для выполнения запроса, которые могут включать запросы к базе данных, взаимодействие со сторонними службами или другие внутренние службы.

Внутренние службы обрабатывают данные и выполняют запрошенные операции, такие как создание, обновление или извлечение ресурсов. После обработки данных сервер API подго-

тавливает ответ, обычно преобразуя данные в стандартный формат, например JSON или XML. Затем сервер API отправляет ответ обратно клиенту, включая коды состояния, указывающие на результат запроса, например, успех, неудачу или ошибки.

Наконец, клиент получает ответ API и обрабатывает данные соответствующим образом, что может включать обновление пользовательского интерфейса, инициирование дополнительных действий или сохранение данных для дальнейшего использования.

По сути, API упрощают взаимодействие между программными приложениями, позволяя разработчикам создавать и поддерживать приложения более эффективно и интегрироваться с другими сервисами и системами для создания мощных, многофункциональных решений.

🕮 *Теория проектирования излагается по книге Арно Лоре «Проектирование веб-API».*

1. Проектирование API для пользователей

Приведенные ниже примеры иллюстрируют разработку API для интернет-магазина (см. Приложение **А**). Рисунки позаимствованы из книги Арно Лоре «Проектирование веб-API».

1.1. Определение целей API

API – это то, что видят пользователи; представление того, что они могут сделать.

Реализация – это код, работающий позади API. Это то, как на самом деле все делается, но остается невидимым для пользователя.

При разработке API важно иметь глубокие и точные знания о том,

- кто может использовать API;
- что он может делать;
- как он это делает;
- что ему нужно для этого;
- что он получает взамен.

Эти фрагменты информации являются основой вашего API. Они нужны вам для проектирования правильного программного интерфейса.

С помощью вопросов «*Что?*» и «*Как?*» мы примерно описываем, что можно сделать с помощью API (что), и разбиваем их на этапы (как), при этом каждый этап становится целью.

Идентификация целей API заключается не только в том, что с ним можно сделать, но и в том, какими данными можно манипулировать с его помощью.

Изучите входные источники и использование выходных данных, чтобы выявить отсутствующие ответы на вопросы «Кто?», «Что?» или «Как?»

Если действие содержит множество этапов или ответвлений, сосредоточьтесь на главном пути, а после этого проверьте, есть ли изменения, приводящие к появлению новых целей в других путях.

1.2. Избегаем точки зрения поставщика при проектировании API

Будем различать разные категории заинтересованных лиц: потребители API, поставщик API и разработчик API.

Проектировать любой API нужно с точки зрения потребителя, а не поставщика!

Независимо от того, проектируете ли вы свой API с нуля или опираетесь на существующие системы, точка зрения поставщика неизбежно будет проявляться на каждом этапе его проектирования.

Точка зрения поставщика проявляется через

- влияние данных
- влияние кода и бизнес-логики
- влияние архитектуры программного обеспечения
- влияния организации, где работают люди

Данные, код и бизнес-логика, архитектура программного обеспечения и общественная организация формируют коммуникационную структуру компании и, следовательно, могут влиять на проектирование ее API.

Проектирование API с точки зрения поставщика путем прямого предоставления доступа к внутреннему устройству (данные, код и бизнес-логика, архитектура программного обеспечения и человеческая организация) неизбежно приводит к появлению сложных для понимания и сложных в использовании API.

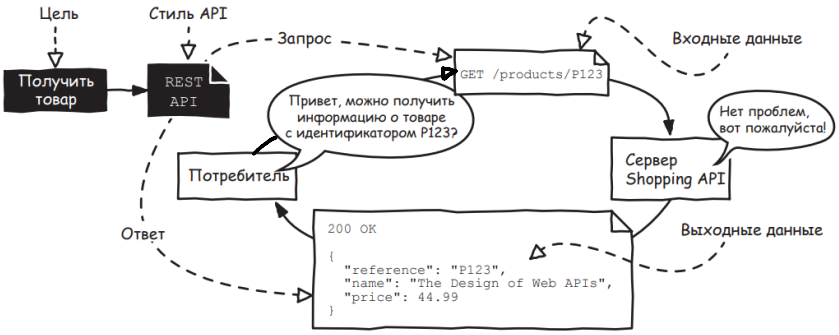
Чтобы потребителям было легко понять и использовать API, он должен проектироваться с точки зрения потребителя (пользователя).

Ориентированный на потребителя список целей API является самой прочной основой для API.

2. Проектирование программного интерфейса (см. Приложение В)

2.1. Знакомство с REST API

Чтобы спроектировать программный интерфейс, мы переносим цели и их входные и выходные данные в API в соответствии со стилем API, как показано на рисунке.



Так как вы уже знакомы с работой протокола HTTP, вспомним основные положения.

- Каким бы ни было его назначение, базовый HTTP-запрос содержит метод HTTP и путь к ресурсу. Метод HTTP указывает, что нужно сделать с ресурсом, который идентифицирует путь.
- За этой первой частью запроса может следовать тело с содержимым ресурса, которое необходимо отправить на сервер, например для создания, обновления или замены ресурса. Это содержимое может быть любого типа: документ в формате JSON, текстовый файл или фотография.

- Ответ HTTP, возвращаемый сервером, всегда содержит код состояния и поясняющую фразу. Как и тело запроса, тело ответа может быть любым.

2.2. Перенос целей в REST API

REST API представляет свои цели, используя протокол HTTP. Цели переносятся в пары типа «ресурс и действие». Ресурсы идентифицируются путями, а действия представлены методами HTTP.

Для переноса целей в REST API необходимо сделать следующие шаги:

1. **Определить ресурсы и связи**
2. **Определить действия, параметры и возвраты**
3. **Спроектировать пути к ресурсам**
4. **Представить действия с помощью протокола HTTP**

2.2.1. Идентификация ресурсов и их связей с таблицей целей API

Сначала надо определить ресурсы. Для этого мы просто анализируем цели API и перечисляем все существительные, к которым применяются основные глаголы целей для идентификации ресурсов.

Чтобы определить организацию ресурсов, мы перечислим цели, в которых упоминается более одного ресурса.

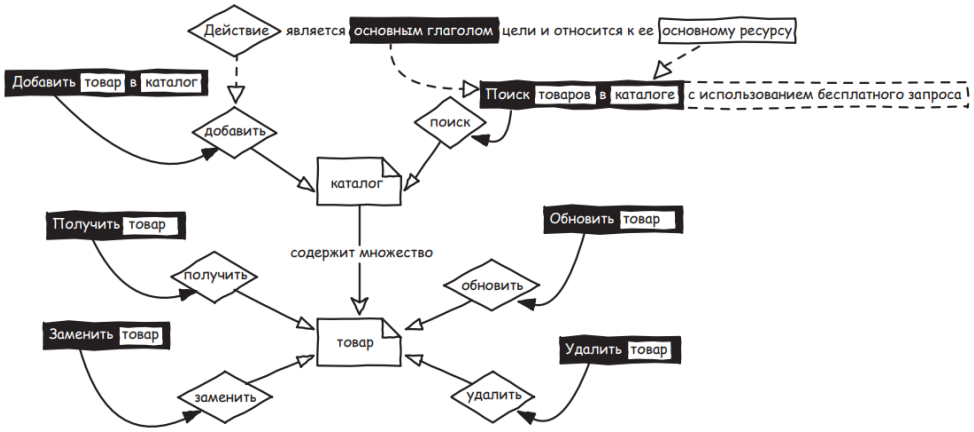
Пусть у нас есть две цели, связанные с более чем одним ресурсом:

- *Добавить товар в каталог*
- *Поиск товара в каталоге*

Здесь *Добавить* и *Поиск* – это действия, а *товар* и *каталог* – ресурсы. Ресурсы могут иметь или не иметь связи с другими ресурсами, а ресурс может содержать какие-либо другие ресурсы того же типа и называться ресурсом-коллекцией или просто коллекцией. В нашем случае ресурс каталога является коллекцией: он содержит ресурсы товара. Возможно, обнаружить связи ресурсов вам поможет концептуальная схема БД.

2.2.2. Идентификация действий, их параметров и результатов с помощью таблицы целей API

REST API представляет свои цели с помощью действий над ресурсами. Чтобы идентифицировать **действие**, мы берем основной глагол цели и связываем его с ресурсом, к которому он применяется, как показано на рисунке.



Идентификация действий

В цели, связанной с более чем одним ресурсом, мы связываем глагол с основным ресурсом (каталогом), который используется или изменяется, а другой ресурс (товар) является только параметром или результатом.

Эти действия могут нуждаться в дополнительных параметрах и могут возвращать некую информацию. Мы уже определили эти параметры и возвращаемые результаты в таблице целей API.

2.2.3. Спроектировать пути к ресурсам

Следующим шагом будет присвоение URL-адресов ресурсу. Есть много возможностей, но, к счастью, соглашения REST дают некоторые рекомендации. В типичном REST API ресурсу будут назначены два шаблона URL. Первый — это множественное число от имени ресурса, например *`/orders`*. Второй — это множественное число от имени ресурса плюс уникальный идентификатор для указания одного ресурса, например *`/orders/{order_id}`*, где *`{order_id}`* — уникальный идентификатор для заказа. Эти два шаблона URL составляют первые **конечные точки (endpoints)**, которые будет поддерживать наш API. Они называются конечными точками просто потому, что они идут в конце URL-адреса, как в *`http://example.com/<endpoint_goes_here>`*.

Путь к ресурсу REST должен быть уникальным. Каждая единица информации однозначно определяется URL – это значит, что URL по сути является первичным ключом для единицы данных. Чтобы повысить удобство использования, связь между ресурсами каталога и товаром можно отобразить в путях наподобие иерархии папок в файловой системе. Каждый ресурс товара – это элемент в коллекции каталога, идентифицируемый как `/catalog`, поэтому мы можем выбрать путь `/catalog/{productId}` для обозначения товара.

Еще пример. Третья книга с книжной полки будет иметь вид `/books/3`, а 35 страница в этой книге — `/books/3/page/35`. Отсюда и получается строго заданный формат. Причем совершенно не имеет значения, в каком формате находятся данные по адресу `/books/3/page/35` – это может быть и HTML, и отсканированная копия в виде jpeg-файла, и документ Microsoft Word. У каждого конкретного ресурса должен быть уни-

кальный URI. Ответственность за то, чтобы у каждого ресурса всегда был свой URI лежит на плечах разработчика сервера.

2.2.4. Представить действия с помощью протокола HTTP

Когда мы ищем товары, мы хотим получить их, поэтому должны использовать HTTP-метод GET в пути `/products`. Чтобы получить только те товары, которые соответствуют некоему запросу, например названию товара или частичному описанию, нужно передать параметр в этот запрос. Параметры HTTP-запроса GET обычно предоставляются в виде параметров запроса в пути, как показано в приведенном ниже листинге.

Листинг 2.1. Примеры параметров запроса

```
GET /example?param1=value1&param2=value2
GET /products?free-query=something
```

- Параметры расположены после знака **?** в конце пути и предоставляются в формате **имя = значение** (например, param1=value1).
- Несколько параметров запроса разделяются символом **&**.

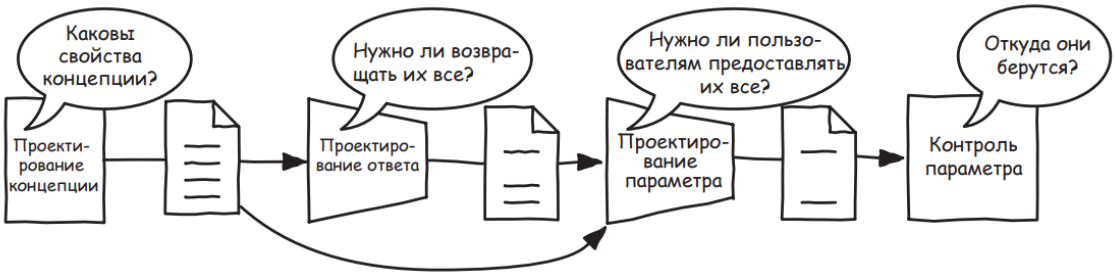
Как только поиск будет завершен, метод запроса GET возвращает ресурсы, соответствующие пути (что включает в себя параметр free-query).

HTTP-методы POST, GET, PUT, PATCH и DELETE, по сути, отображают базовые функции CRUD (создание, чтение, обновление, удаление). Не забывайте, что эти действия выполняются с точки зрения потребителя; например, если вы выполните действие `DELETE /orders/123`, это не означает, что заказ 123 будет фактически удален из базы данных, содержащей заказы. Такие действия могут просто изменить статус этого заказа на CANCELED.

Эти HTTP-методы также должны использоваться для представления более или менее таковых действий CRUD. Порой проектировщикам REST API бывает трудно выбрать, какой HTTP-метод соответствует действию, которое явно не отображается в функцию CRUD. В таблице П.1 Приложения В приведены примеры действий, которые могут помочь вам выйти за рамки CRUD.

2.3. Проектирование данных API

Идентифицированные вами ресурсы, параметры и возвращаемые результаты описаны нечетко. Как спроектировать эти элементы данных? Процесс проектирования описан на рисунке.

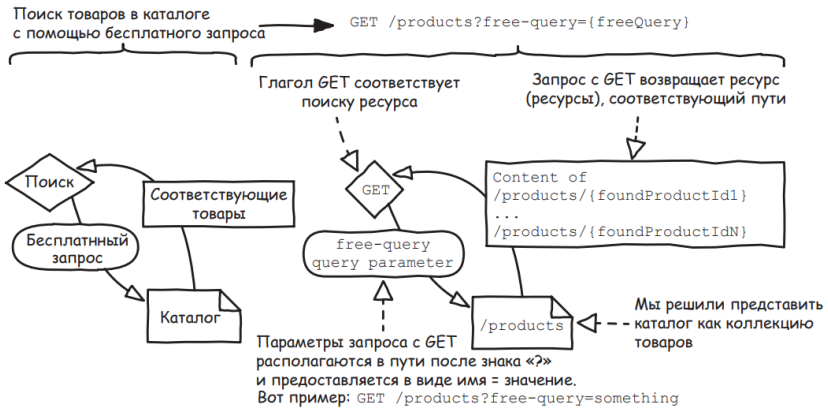


Под концепцией в данном случае понимается некоторая бизнес-сущность, ресурс.

После того как мы спроектировали основные концепции, можем спроектировать параметры и ответы, адаптируя их. Кроме того, мы должны гарантировать, что потребители смогут предоставить все данные, необходимые для использования API. Независимо от типа API мы начинаем проектировать данные точно так же, как любое программируемое представление какой-либо сущности – как таблицу базы данных, структуру или объект.



Действия с ресурсом



2.3.1. Проектирование ресурсов

Все сущности, которые мы определили и превратили в ресурсы REST, будут передаваться между потребителем и поставщиком через параметры и ответы.

Проектирование начнем с перечисления свойств структуры данных и дадим каждому свойству имя. Например, у товара могут быть свойства `reference`, `name` и `price`. Также может быть полезно сообщить покупателю, когда товар был добавлен в каталог (`dateAdded`) и есть ли он в наличии или нет (`unavailable`).

Эта работа вам хорошо знакома из курса «Базы данных». Спроектированные свойства ресурсов в дальнейшем будут представлены в формате JSON.

Листинг 2.2. Пример товара в формате JSON

```
{
  "id": "S789",
  "name": "a product",
  "price": 9.99,
  "dateAdded": "2018-01-02",
  "definitelyOutOfStock": false,
  "supplier": {«id": "R123-456",
               "name": "a supplier"
  }
}
```

Товар состоит из обязательных и необязательных свойств. Некоторые из них (например, `id`, `price`, `definitelyOutOfStock` и `dateAdded`) относятся к базовым типам (например, строка, число, логическое значение или дата). Также могут быть и сложные свойства, например `supplier`, которое является объектом. Свойство также может быть объектом, содержащим свойства или массив.

2.3.2. Проектирование ответов от ресурсов

Один и тот же ресурс может появляться в разных контекстах: *Найти товар, Добавить товар, Получить товар*. Во всех трех случаях действия возвращают *Товар*, это ответ ресурса, который представлен в разных контекстах.

Если действия «Добавить товар» и «Получить товар» должны вернуть товар целиком, действие «Найти товар» может вернуть только ссылку, имя, цену и название поставщика в виде `supplierName`.

При проектировании ответов мы не должны слепо сопоставлять действию манипулируемый ресурс. Мы должны адаптировать их к контексту, удаляя или переименовывая свойства, а также корректируя структуру данных.

2.3.3. Проектирование параметров

Когда мы добавляем, обновляем или заменяем ресурс, мы передаем некую информацию о ресурсе в качестве параметра.

Но из чего состоит этот параметр или параметры. Например, параметр информации о товаре может быть разным:

- При создании товара ссылка на него генерируется серверной частью, поэтому потребителю не нужно предоставлять ее при добавлении товара.
- Но нам эта ссылка нужна для обновления или замены товара (обратите внимание, что ссылка будет передана в путь в качестве его параметра: /products/{id}).
- Во всех этих случаях потребителю не нужно предоставлять свойство `supplier.name` – нужна только ссылка. У серверной части есть способ найти имя поставщика на основе ссылки. Поэтому, чтобы упростить организацию данных, можно удалить объект `supplier` и заменить его на `supplierReference`, то есть на его `id`.
- `dateAdded` также генерируется серверной частью при добавлении товара в каталог, поэтому нам это тоже не нужно.

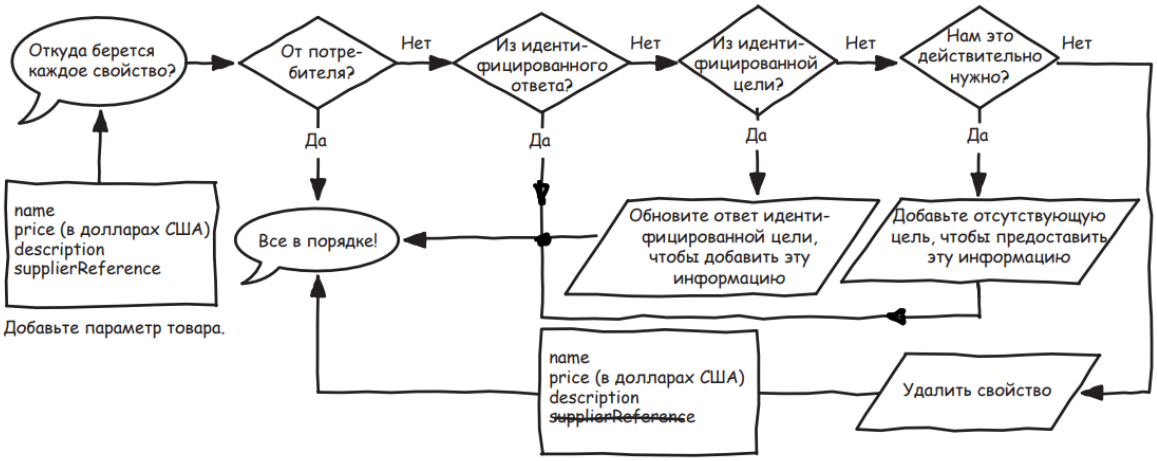


Параметр должен предоставлять необходимые данные, но не более того. Он не должен включать в себя данные, которые обрабатываются исключительно серверной частью.

Действуя таким образом, мы гарантируем, что параметр информации о товаре содержит только те данные, которые абсолютно необходимы в каждом контексте.

2.3.4. Проверка параметров

При добавлении товара в каталог потребители должны иметь возможность легко предоставлять такие данные, как *название*, *цена* и *описание*. А что насчет *supplierReference*? Откуда потребители могут знать такую ссылку?



Потребители должны иметь возможность предоставлять все данные параметра либо потому, что они сами располагают сведениями, либо по той причине, что они могут извлечь их из API. Если данные не могут быть предоставлены, это может быть признаком отсутствующей цели или точки зрения поставщика. Поэтому мы должны еще раз проверить, что все необходимые данные могут быть предоставлены потребителем. Этот процесс проверки, показанный на рисунке, гарантирует, что потребители всегда смогут предоставить данные параметров и что в API нет никаких пробелов.

В данном случае потребители уже могут знать о *supplierReference,* поскольку

- Эта информация указана на этикетке товара.
- Или ее можно получить из уже спроектированного нами ответа.
- Она также может исходить из другой цели. В этом случае нам просто нужно обновить ответ, чтобы добавить эту информацию и предоставить ее потребителю.
- Возможно, мы просто пропустили цель. В этом случае нам нужно будет добавить ее в таблицу целей API и обработать, как любую другую цель, определив, кто может ее использовать, ее входные и выходные данные и спроектировав ее программируемое представление. Или мы могли бы просто понять, что нам это не нужно.

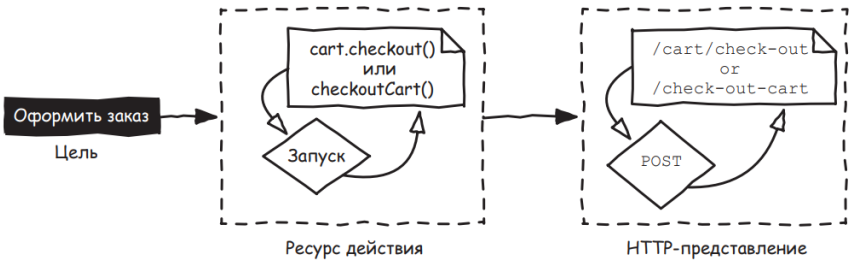
Главное - параметры нужно тщательно проверять! Вы можете обнаружить какие-то недостающие функции, когда познакомитесь с деталями разработки API.

2.4. Достижение баланса при решении проблем проектирования

Иногда вы можете изо всех сил пытаться найти представление цели, которая соответствует выбранной модели API, или получить представление, соответствующее модели, но оно будет не таким удобным для пользователя, как вы ожидали. Иногда идеального представления не существует, поэтому вам, как проектировщику API, придется искать компромисс.

2.4.1. Примеры компромисса

Когда пользователи покупают товары, в конце им нужно оформить заказ. Как представить такую цель, когда неочевидно, как перенести ее в пару «путь–HTTP-метод»? Когда проектировщику не удается сопоставить действие над ресурсом REST с каким-либо HTTP-методом, первым вариантом часто является создание ресурса действия.



Ресурс действия – это не вещь, обозначаемая существительным, а действие, обозначаемое глаголом. Это просто функция. Ресурс действия можно рассматривать как метод класса и, следовательно, он может быть представлен как подресурс ресурса. Если мы решим представить ресурс корзины с помощью пути /cart, то

- метод `cart.checkout()` может быть представлен путем `/cart/check-out`.
- Но мы также могли бы рассматривать его как отдельную функцию `checkoutCart()` и, соответственно, создать ресурс действия `/check-out-cart`.

В обоих случаях мы используем HTTP-метод POST для запуска ресурса действия.

HTTP-метод POST – метод по умолчанию, который используется, когда никакой другой метод не подходит.

Ресурс действия абсолютно не соответствует модели REST, но он работает и полностью понятен потребителям.

2.5. Влияние ограничений REST на проектирование API

Существуют разные стили программных интерфейсов (SOAP, REST, gRPC или GraphQL), позволяющие программному обеспечению обмениваться данными удаленно. У каждого из них есть, были или будут свои особенности, плюсы и минусы, и каждый из них, очевидно, сможет создавать API особого типа. Но, независимо от стиля, проектирование API требует в основном одного и того же мышления.

2.5.1. Знакомство с архитектурным стилем REST

(см. лекции)



ЗАДАНИЕ

Будем исходить из предположения, что вы разрабатываете веб-приложение в рамках курсового проектирования. Клиентская и серверная части общаются через REST API.

Вам требуется разработать только часть всего веб-API. Эта часть должна включать в себя все возможные действия с ресурсами: просмотр, добавление, обновление, удаление, … Пользователей, как и ресурсов, должно быть не меньше двух. Для выбора действий воспользуйтесь уже разработанной диаграммой вариантов использования (ВИ). При проектировании обязательно ссылайтесь на идентификатор ВИ (UC).

Предварительное проектирование REST API

На этапе предварительного проектирование необходимо решить следующие задачи:

1. Определить, какие ресурсы должны быть доступны.
2. Назначить URL-адреса этим ресурсам.
3. Решить, какие действия клиенту следует разрешить выполнять с этими ресурсами.
4. Выяснить, какие фрагменты данных требуются для каждого действия и в каком формате они должны быть.

1. Создайте таблицу целей API (см. Приложение А).

№	Пользователи (Кто)	Действия (Что)	Этапы (Как)	Входные данные (источник)	Выходные данные (использование)	Цели
	Кто пользователи?	*Что они могут делать?*	*Как они это делают?*	*Что им нужно? Откуда это?*	*Что они получают? Как это используется*	*Переформулируйте действия+входные данные+результаты*
1	**Клиенты**	Купить товары	Найти товары	Каталог (управление каталогом) Запрос (предоставляется пользователем)	Товары, соответствующие запросу	Найти товары в каталоге с помощью запроса
			Добавить товар в корзину	Товар (найти товары) Корзина (принадлежит пользователю)	Ничего не получают	Добавить товар в корзину
			Оформить заказ	Корзина	Заказ (управлять заказом)	Оформить заказ
			Управлять заказом	…	…	…
2	**Администратор**	Управлять каталогом	Добавить товар в каталог	Каталог (принадлежит пользователю) Товар (предоставляется пользователем)	Ничего не получают	Добавить товар в каталог
			Заменить			
			….			

2. Устраните влияние поставщика при проектировании API. Результаты изменений занесите в таблицу

Влияние	Результаты изменения
1. Данных	
2. Кода и бизнес-логики	
3. Архитектуры ПО	
4. Организации	

3. Соотнесите полученную таблицу целей API с полученной в предыдущей лабораторной работе диаграммой вариантов использования. Сформируйте таблицу с целевыми вариантами использования (UC) и этапами их выполнения (в формате расширения и включения).

Субъект (Actor)	Целевой вариант использования (UC - Use Case)	Расширения и включения (Include, Extend)
Пользователь (Менеджер, Покупатель)	UC 1. Войти в систему	UC 1.1 Регистрация (расширение) UC 1.2 Аутентификация (расширение)
	UC 2. Найти товар	UC 2.1 Посмотреть каталог (включение) UC 2.2 Посмотреть товар (расширение)
Сотрудник	UC 3. Добавить новый товар	UC 1.2 Аутентификация (включение)
Покупатель	UC 5. Купить товар	UC 5.1 Сделать заказ (включение) UC 5.1 Добавить товар в корзину (включение) UC 5.2 Применить скидку (расширение)

4. Сформируйте обновленный и окончательный вариант таблицы целей API.

5. Сопоставьте варианты использования (ВИ) системы с маршрутами и конечными точками. Маршрут (Route) — это URL-адрес, который направляет запрос к определенным конечным точкам с помощью HTTP-методов. Маршрут может иметь несколько конечных точек. **Конечная точка** (Endpoint) — это непосредственно обращение к маршруту конкретным HTTP-методом, чтобы выполнить определенную задачу и вернуть данные с сервера клиенту. Результаты сопоставления ВИ с маршрутами и конечными точками сведем в таблицу.

Субъект (Actor)	Вариант использования (UC - Use Case)	Маршрут (Route)	HTTP-метод
Пользователь (Сотрудник, Покупатель)	UC-1. Посмотреть каталог товаров	/product	GET
	UC-2. Найти товар	/product?free-query={free query}	GET с параметрами фильтрации
Сотрудник	UC-6. Изменить товар	/product/{id}	PUT
…	…	…	…

Ресурсы обычно имеют различные конечные точки доступа и несколько методов для каждой конечной точки. На одной и той же странице обычно описывается общий ресурс и ряд конечных точек для доступа к ресурсу.

6. Составьте структуру данных, которая будет отражать сведения о товаре (Product) и поставщике (Provider). Сначала определимся с полями этих сущностей и их типами данных:

Ресурс	Поле	Описание свойства	Тип данных	Обязательно
product (Товар)	id	Идентификатор товара	integer	да
	name	Название товара	string	да
	category	Категория товара	string	да
	supplier	Поставщик товара	object	да
	price	Цена товара	number	да
	quantity	Количество единиц товара	integer	да
	definitelyOutOfStock	Наличие товара	boolean	нет
	dateAdded	Когда товар был добавлен в каталог	date	нет
supplier (Поставщик)	id	Идентификатор поставщика	integer	да
	…	…	…	…

Учтите, что свойства могут быть и объектами. Например, свойство *supplier*. Смотри Листинг 2.2. Свойство также может быть объектом, содержащим свойства или массив.

Обратите внимание, что обязательный или необязательный статус свойства может варьироваться в зависимости от контекста.

На этом предварительное проектирование REST API выполнено, можно переходить к формированию спецификации.

ТРЕБОВАНИЯ К ОТЧЕТУ

- Описание предметной области
- Все таблицы согласно заданию на проектирование



КОНРОЛЬНЫЕ ВОПРОСЫ

1. Что еще, кроме REST API, можно использовать для организации связи между приложениями?
2. Какие типы API существуют? (stoplight.io)
3. Приведите примеры использования API (stoplight.io)

4. Что такое спецификация API?
5. Что такое конечная точка API?
6. Какие задачи решаются на этапе предварительного проектирования API?
7. Какие команды HTTP являются идемпотентными?
8. Существует ли клиентский веб API? (Web API - Wikipedia (turbopages.org))
9. Из чего состоит серверный веб-API ? (Web API - Wikipedia (turbopages.org))
10. Как в REST API передаются данные на сервер? (строка запроса Query String и тело запроса)
11. Из каких шагов складывается обработка API-запросов? (маршрутизация запроса в соответствующую службу, проверка данных, логика, …)
12. Продолжите предложение «При разработке API важно иметь глубокие и точные знания о том,
    - кто может использовать API;
    - …

ИСТОЧНИКИ

Рекомендуемая литература

- Арно Лоре «Проектирование веб-API»

Сайты

- Курс по документированию REST API | learnapidoc-ru (starkovden.github.io)
- Проектирование Web API | Sam's Workbook (gasches.cc)
- Руководство OpenAPI Шаг 1 Объект "openapi" | learnapidoc-ru (starkovden.github.io)
- Подходы к проектированию RESTful API / Хабр (habr.com)
- Сергей Константинов. API (twirl.github.io)
- Проектирование RESTful Web API (luisrei.com)
- Знакомство со спецификациями OpenAPI и Swagger | learnapidoc-ru (starkovden.github.io)
- Учебник OpenAPI шаг за шагом learnapidoc-ru (starkovden.github.io)
- Шаблоны проектирования API для веб-служб REST - DEV Community
- Проектирование CRUD API и рекомендации по CRUD API (stoplight.io)
- Как читать документацию по API | Авто стоп-сигнал (stoplight.io)
- Что такое API?(stoplight.io)
- Оцените свою документацию по API с помощью этого руководства | (stoplight.io)
- Каталоги API: чем они хороши? | (stoplight.io)
- Продукты API нуждаются в каталогах API | (stoplight.io)
- Использование схемы JSON для пользовательских ответов API | (stoplight.io)
- Редактор JSON - редактируйте код JSON за секунды (jsononline.net)
- Stoplight | Stoplight
- https://stoplight.io/welcome
- Что такое конечная точка API? | Облачная вспышка (cloudflare.com)
- Как создать свой Swagger на REST-приложение в Google Colab (babok-school.ru)
- Проектируем и документируем REST: спецификация OpenAPI (babok-school.ru)
- YAML Ain't Markup Language (YAML™) версия 1.2.2
- Как создать свою спецификацию OpenAPI | ЕСКС (gosuslugi.ru)
- Полное руководство по разработке API | AppMaster
- Рекомендации по проектированию веб-API - Azure Architecture Center | Microsoft Learn
- OpenAPI 3.1.0 (русская редакция) - maksyutin.ru / Разработка ПО: взгляд изнутри

- Swagger (OpenAPI 3.0) / Хабр (habr.com)
- 10 примеров для изучения модуля JSON в Python (nuancesprog.ru)
- Визуальное проектирование API для OpenAPI | Поддержка конструктора схем API JSON | Авто стоп-сигнал (stoplight.io) 14-дневная бесплатная пробная версия.
- OpenAPI Generator - Генерация кода по спецификации (yandex.ru)
- Учебное пособие по генератору OpenAPI (практическое) (apidog.com)
- JMeter как инструмент для автоматизации функционального тестирования / Хабр (habr.com)
- Шаг 1 "Описание ресурса (Описание API)" | learnapidoc-ru (starkovden.github.io)
- Шаг 2 "Конечные точки и методы (Описание API)" | learnapidoc-ru (starkovden.github.io)
- Шаг 3 "Параметры (Описание API)" | learnapidoc-ru (starkovden.github.io)
- Шаг 4 "Пример запроса (Описание API)" | learnapidoc-ru (starkovden.github.io)
- 5 "Пример и схема ответа (Описание API)" | learnapidoc-ru (starkovden.github.io)
- Собираем все вместе (Описание API) | learnapidoc-ru (starkovden.github.io)
- О модуле "Спецификация OpenAPI и Swagger" | learnapidoc-ru (starkovden.github.io)
- Web API - Wikipedia (turbopages.org)
- Разработка web API / Хабр (habr.com)

Видео

- Введение в REST API за 7 минут | Listen IT | Дзен (dzen.ru)
- Что такое REST API. Создаем API с нуля на Express
- Знакомство со Swagger документацией. Разбор основных методов запросов
- Как использовать swagger
- Первый опыт в OpenAPI / Swagger - это PetStore проект
- OpenAPI и Swagger Editor - своё описание REST API с нуля
- OpenAPI Generator - Генерация кода по спецификации
- (11) Мир мобильного аналитика. Surf BA School - YouTube
- (11) Проектирование API. Теория. Surf BA School - YouTube
- (11) Проектирование API. Практика. Surf BA School - YouTube
- (11) Составление спецификаций. Теория. Surf BA School - YouTube
- (11) Составление спецификаций. Практика. Surf BA School - YouTube
- https://yandex.ru/video/preview/7438056860438307276

Инструменты

- Введение | AppMaster
- Решения для проектирования и управления API (stoplight.io)
- Авто стоп-сигнал (stoplight.io)
- Apidog An integrated platform for API design, debugging, development, mock, and testing

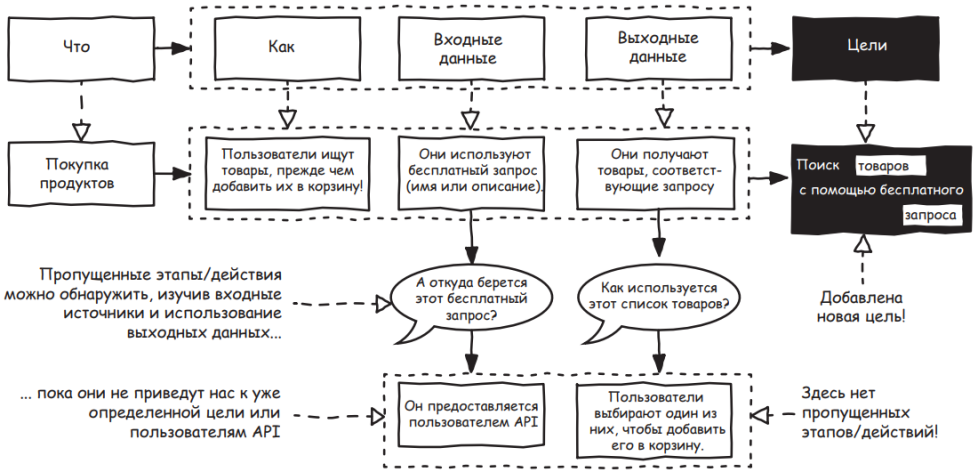
ПРИЛОЖЕНИЕ

A. Проектирование API для пользователей

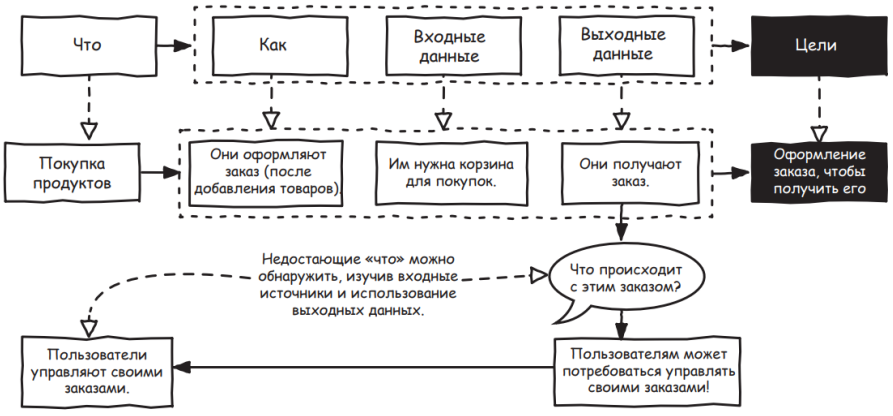


Выявляем недостающие цели.

1. Как пользователь получает товар, чтобы добавить его в корзину?



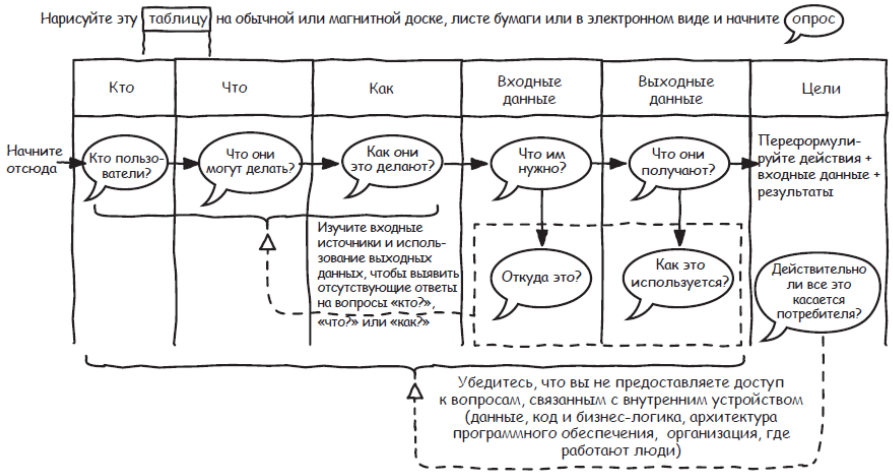
2. Как пользователи управляют своими заказами?

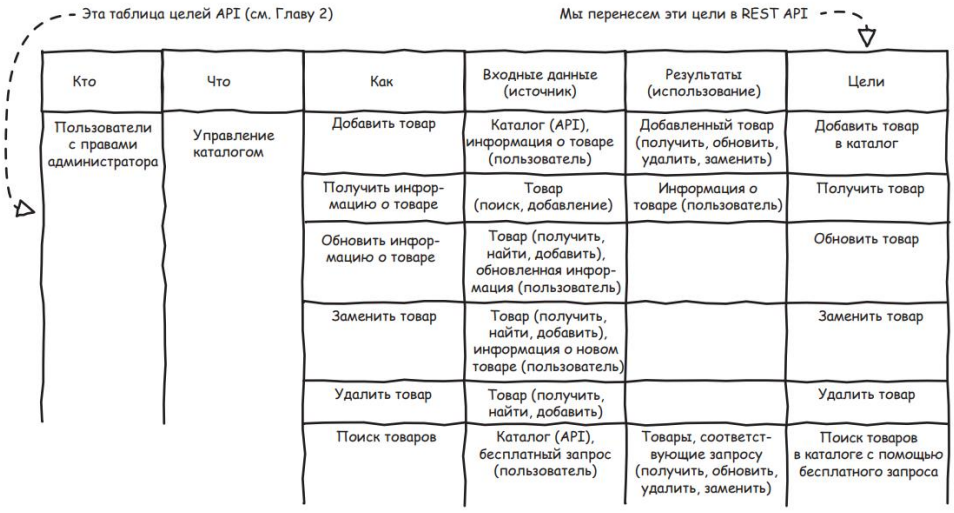


Список целей API онлайн-магазина с новым действием «Пользователи управляют своими заказами»



Окончательный и обновленный вариант нашей таблицы целей API



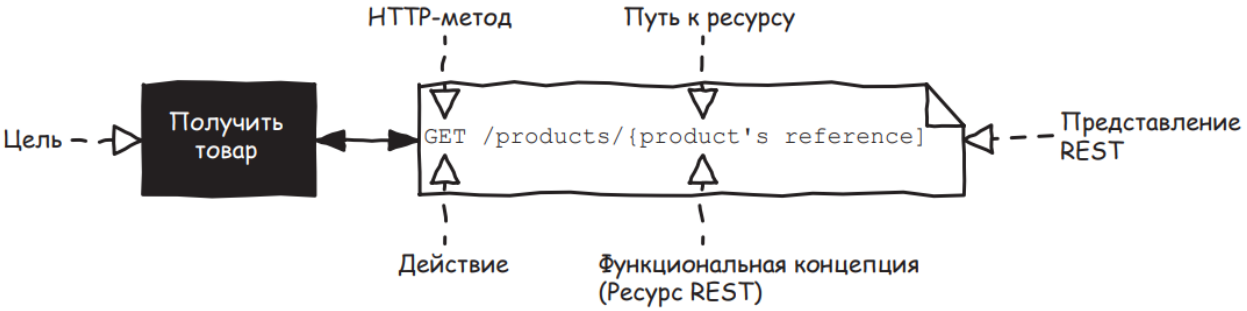


Кто	Что	Как	Входные данные (источник)	Результаты (использование)	Цели
Пользователи с правами администратора	Управление каталогом	Добавить товар	Каталог (API), информация о товаре (пользователь)	Добавленный товар (получить, обновить, удалить, заменить)	Добавить товар в каталог
		Получить информацию о товаре	Товар (поиск, добавление)	Информация о товаре (пользователь)	Получить товар
		Обновить информацию о товаре	Товар (получить, найти, добавить), обновленная информация (пользователь)		Обновить товар
		Заменить товар	Товар (получить, найти, добавить), информация о новом товаре (пользователь)		Заменить товар
		Удалить товар	Товар (получить, найти, добавить)		Удалить товар
		Поиск товаров	Каталог (API), бесплатный запрос (пользователь)	Товары, соответствующие запросу (получить, обновить, удалить, заменить)	Поиск товаров в каталоге с помощью бесплатного запроса

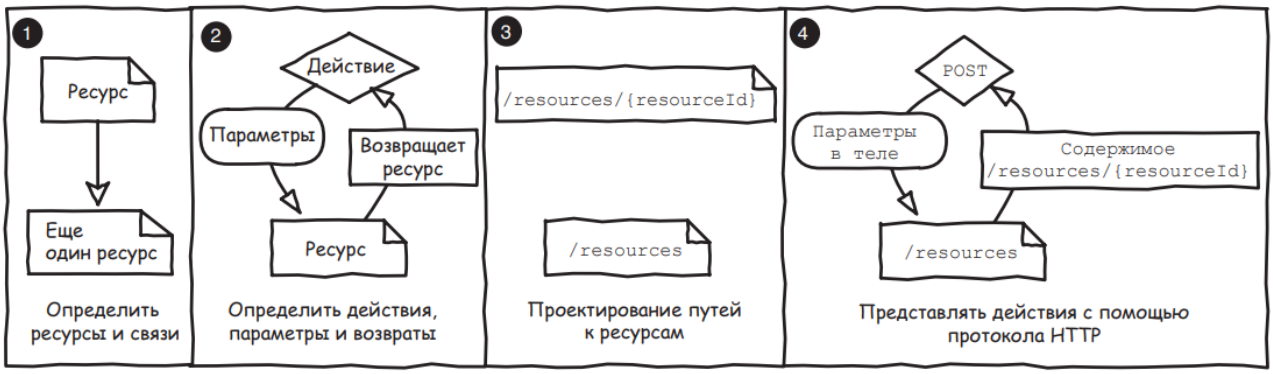
B. Проектирование программного интерфейса

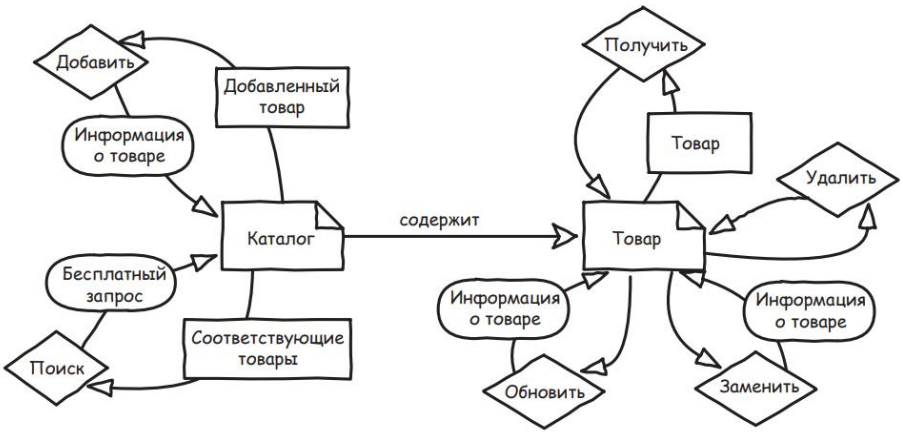


Базовая структура HTTP-запроса и ответа



Отображение цели в HTTP-запрос

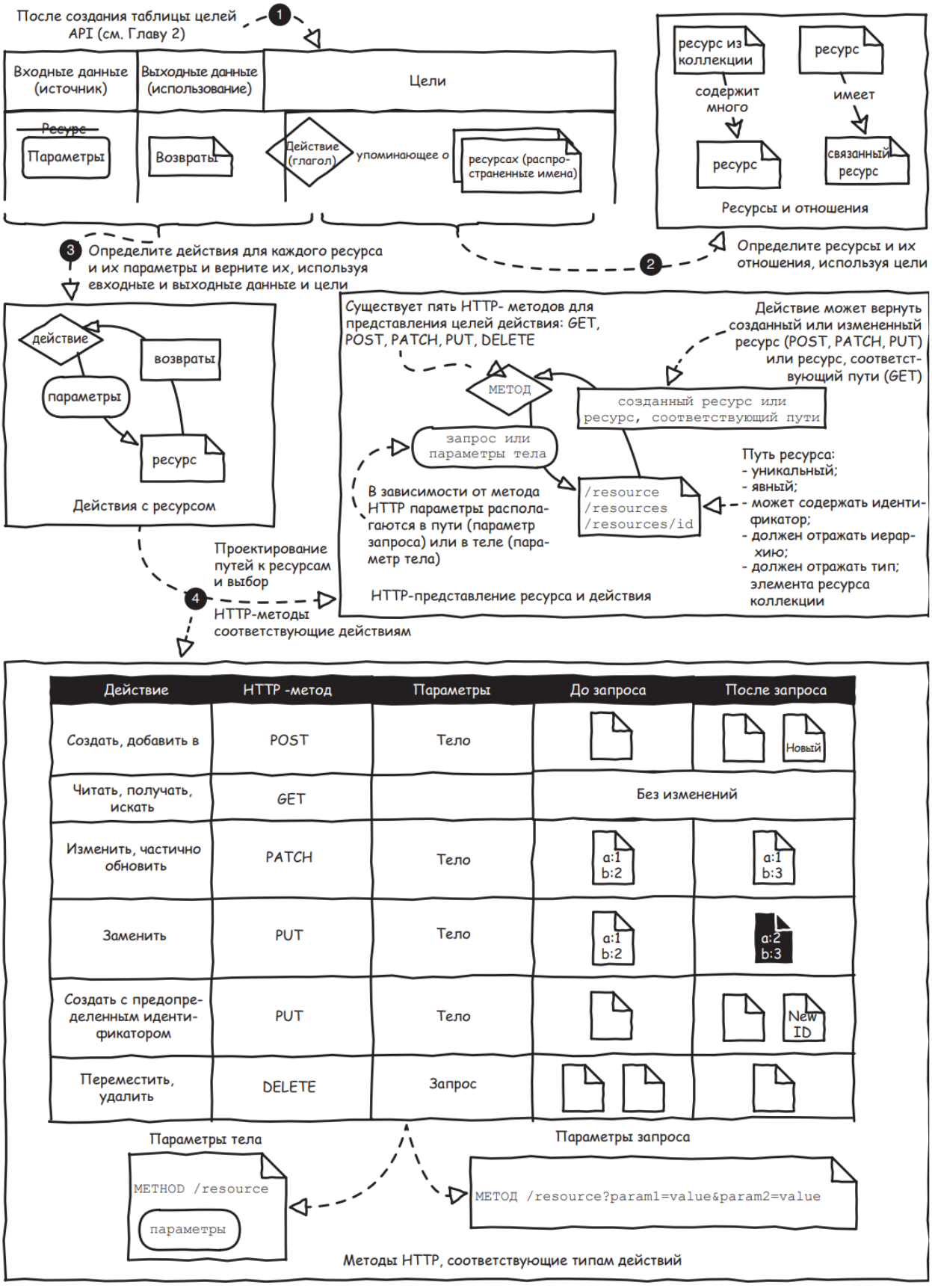




Все идентифицированные ресурсы и действия

Таблица П.1. HTTP-методы за пределами CRUD

HTTP-метод	Действие
POST (и PUT в создании)	Создать клиента, добавить блюдо в меню, заказать товары, запустить таймер, сохранить запись в блоге, отправить сообщение в службу поддержки, подписаться на услугу, подписать договор, открыть счет в банке, загрузить фотографию, поделиться статусом в социальной сети и т. д.
GET	Прочитать клиента, найти французский ресторан, найти новых друзей, извлечь открытые аккаунты за последние три месяца, загрузить подписанный контракт, отфильтровать самые продаваемые книги, выбрать черно-белые фотографии, перечислить друзей и т. д.
PATCH/PUT	Обновить клиента, заменить товар в заказе, поменять места в самолете, изменить способ доставки заказа, изменить валюту заказа, изменить лимит дебетовой карты, временно заблокировать кредитную карту и т. д.
DELETE	Удалить клиента, отменить заказ, закрыть дело, завершить процесс, остановить таймер и т. д.



REST API и шпаргалка по HTTP

C. Вопросы и ответы

Что такое вызов API?

Интерфейсы прикладного программирования (API) — это способ взаимодействия одной программы с другой. Вызовы API — это среда, с помощью которой они взаимодействуют. Вызов API, или запрос API, — это сообщение, отправленное на сервер с просьбой к API предоставить услугу или информацию.

Куда направляется вызов API?

Вызовы API передаются от клиента к конечной точке API. Конечные точки API — это места, куда ведут вызовы API — чаще всего это веб-приложение и сервер. Мобильный клиент, например, генерирует вызов API, который направляется в конечную точку API, сервер. Сервер получает вызов API, обрабатывает его, выполняет запрос и отправляет ответ.

Вызовы API направляются на универсальный код ресурса (URI).

URI — это стандартизированный способ идентификации ресурса, подобно тому, как номер телефона идентифицирует телефонную линию. Идентифицированный ресурс может быть веб-сайтом, приложением, сервером, контактом электронной почты или даже реальным элементом.

Для веб-API, как правило, универсальный код ресурса (URI) представляет собой унифицированный указатель ресурса (URL). URL-адрес — это тип универсального кода ресурса (URI) для идентификации расположений в Интернете, таких как веб-сайт или сервер. URL-адрес должен включать протокол прикладного уровня, такой как HTTP, используемый для доступа к нему. Адреса веб-страниц записываются в виде URL-адресов, например «https://www.cloudflare.com/learning». Конечные точки API также являются URL-адресами. Большинство веб-API используют протокол HTTP, поэтому он включен в URL-адрес конечной точки API.

Но url — это именно universal resource locator, универсальный идентификатор ресурса, а не ссылка в интернете.

Что такое конечная точка API?

Интерфейс прикладного программирования (API) — это способ, с помощью которого приложение запрашивает службу у другого приложения. API-интерфейсы позволяют разработчикам избежать перестроения уже существующих функций приложения. Конечная точка API — это место, где выполняются эти запросы (известные как вызовы API). Они существуют в программном, а не в аппаратном обеспечении.

API-серверы и API-клиенты

API размещаются на одном или нескольких серверах — специализированных компьютерах, которые хранят данные и выполняют программы. Каждый сервер «обслуживает» данные, контент и функциональность программного обеспечения другим устройствам через Интернет. Конечная точка API почти всегда размещается на сервере.

Другим концом API-соединения является API-клиент — сущность, запрашивающая сервисы у API. Некоторые называют это «пользователем» API, хотя большинство вызовов API автоматизированы.

Как API-клиент узнает конечную точку сервера?

Чтобы API можно было использовать, у него должна быть документация. В документации указано, среди прочего, какие типы запросов будет принимать API, что может делать API, как он форматирует

свои ответы и каковы его конечные точки. Разработчики могут просматривать документацию API и использовать эту информацию при создании своих приложений.

Как API используют URL-адреса?

Унифицированные указатели ресурсов (URL-адреса) используются для различных целей в Интернете, включая поиск веб-страниц. URL-адреса также указывают на конечные точки API.

На сервере API может размещаться одна или несколько конечных точек API, что означает, что он будет принимать и обрабатывать вызовы, направленные на URL-адреса этих конечных точек. Клиенты API также должны иметь URL-адрес, чтобы сервер API знал, куда отправлять свои ответы точно так же, как Бобу и Алисе нужен номер телефона для телефонных звонков между ними. Разработчики задают этот URL-адрес при сборке своих приложений.

URL-адрес всегда включает в себя протокол прикладного уровня, например HTTP, используемый для его доступа. Большинство веб-API используют протокол HTTP, поэтому он включен в URL-адрес конечной точки API.

Аутентификация — это процесс проверки личности. Точно так же, как существует несколько способов проверки подлинности пользователя в системе, существует четыре основных способа, с помощью которых конечные точки API могут обеспечить проверку подлинности:

1. **Ключ API:** API-клиенту назначается ключ или уникальная строка символов, которая известна только ему и API-сервису. Когда клиент API отправляет вызов API в конечную точку сервера, он включает ключ, чтобы сервер знал, откуда поступил вызов.
2. **Базовая аутентификация (логин и пароль):** Как и в случае с ключом, клиент API устанавливает имя пользователя и пароль в службе API и включает эти учетные данные в вызовы API.
3. **Токен OAuth:** Вместо того, чтобы требовать аутентификацию от клиента, сервер API может получить маркер проверки подлинности от доверенного сервера проверки подлинности с помощью протокола OAuth.
4. **Взаимный** TLS: TLS — это протокол, который создает аутентифицированное соединение между клиентом и сервером при загрузке веб-страниц. Он также может аутентифицировать обе стороны интеграции API.

Во многих случаях взаимный TLS является наиболее эффективным методом проверки подлинности. Во-первых, он аутентифицирует как конечную точку, так и клиента, а не только клиента, поэтому обе стороны могут быть уверены, что они получают данные из законного источника. Он также использует закрытые ключи, которые никогда не передаются между конечными точками и, следовательно, не могут быть перехвачены при передаче. С другой стороны, ключи API, пароли и токены могут быть продублированы или украдены.

Что такое веб-API?

Все, что связано с компьютерным кодом, может иметь API, от операционных систем до программных библиотек. Веб-API предназначен специально для использования веб-приложениями, доступ к которым осуществляется через Интернет.

Веб-API невероятно важны для современного Интернета. Почти все пользовательские приложения полагаются на API для работы. Вся философия разработки программного обеспечения основана на использовании API — одной из таких философий является JAMstack, где JAM расшифровывается как JavaScript, API, разметка. Другим примером является микросервисная архитектура, которая использует API для вызова различных функций, составляющих приложение. Даже приложения, созданные без этих подходов, обычно полагаются на API.

Часть работы с REST API — это создание описаний работы API: информации о ресурсах, параметрах запросов, возвращаемых данных, конечных точках и других важных вещах. Чтобы автоматизировать это описание, сделать его структурированным и прозрачным, разработчики используют Swagger. А системные аналитики, в свою очередь, с его помощью формируют требования к IT-системам.

Что такое FastAPI?

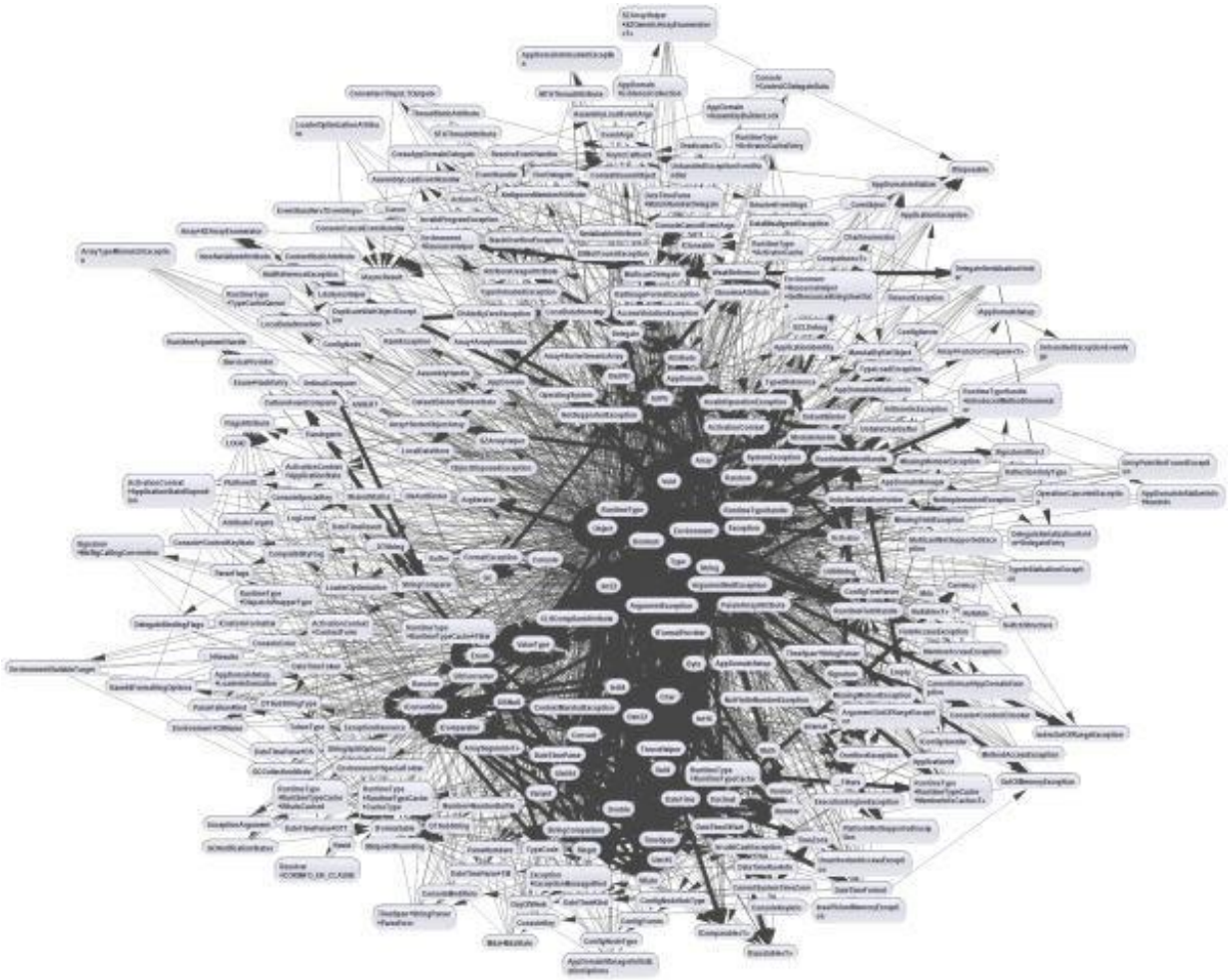
FastAPI — веб-фреймворк для создания API, написанный на Python. Один из самых быстрых и популярных (после Django и Flask) веб-фреймворков, написанных на Python.

В классических языках программирования нет готовых возможностей для сохранения структурированных объектов, но, т.к. структуры данных напрямую лежат в памяти, есть легкий способ сохранения и восстановления данных напрямую.

В Java данные элементов одного объекта произвольно разбросаны по памяти JVM, поэтому даже если бы была возможность сохранения структуры объекта целиком, сохранить данные бы не получилось, поэтому единственно возможный путь в Java – это поэлементное сохранение данных элементарных типов из которых состоит объект.

Само понятие сериализации никак не привязано к формату данных, в который будут сохранены данные, поэтому вне зависимости от того, какой результат будет получен – бинарный файл с собственной структурой, формат XML, JSON или даже текстовый файл – все это будет сериализацией.





Проектирование Web API | Sam’s Workbook (gasches.cc)

Рекомендации по проектированию веб-API - Azure Architecture Center | Microsoft Learn

критерий проверки, который мы используем при проектировании *Web API*, является наличие только **2-х URL на каждый ресурс**. Давайте смоделируем API для простого объекта или ресурса - собаки (dog), и создадим для него *Web API*.

Первый URL - для коллекции, второй - для конкретного элемента в коллекции:

- `/dogs`
- `/dogs/1234`

Сокращение API до такого уровня также стимулирует к исключению глаголов в базовом URL. **Для работы с коллекциями и элементами используйте HTTP-глаголы.**

Для нашего ресурса `dog` имеется два базовых URL, обозначенных существительными, и теперь мы можем работать с ними используя HTTP-глаголы. Наши HTTP-глаголы это: *POST*, *GET*, *PUT* и *DELETE* (что соответствует аббревиатуре *CRUD* - **C**reate-**R**ead-**U**pdate-**D**elete).

Имея два ресурса (`/dogs` и `/dogs/1234`) и четыре HTTP-глагола мы обладаем широкими возможностями, которые интуитивно понятны прикладному разработчику.

В итоге:

- **Используйте два базовых URL на каждый ресурс.**
- **Не используйте глаголы в базовом URL.**
- **Для работы с коллекциями и элементами используйте HTTP-глаголы.**

Ресурс не обязательно должен быть основан на одном физическом элементе данных. Например, ресурс заказа может быть реализован внутри системы с использованием нескольких таблиц в реляционной базе данных, однако представлен в клиентском приложении как единая сущность.

Избегайте создания API, которые просто отражают внутреннюю структуру базы данных. Цель REST — моделировать сущности и операции, которые приложение может выполнять в этих сущностях. Внутренняя реализация должна быть скрыта от клиента.

Сущности часто группируются в коллекции (заказов, клиентов). Коллекция — это отдельный ресурс из элемента в коллекции и должен иметь свой собственный URI. В результате отправки HTTP-запроса GET на URI коллекции возвращается список элементов в коллекции. Каждый элемент в коллекции также имеет свой собственный универсальный код ресурса (URI). HTTP-запрос GET к URI элемента возвращает подробные сведения об этом элементе.

Также следует продумать связи между разными типами ресурсов и способы предоставления этих связей. Например, **`/customers/5/orders`** может представлять все заказы для клиента 5. Вы также можете пойти в другом направлении и представить связь между заказом и конкретным клиентом посредством универсального кода ресурса, например **`/orders/99/customer`**. Однако чрезмерное расширение этой модели может вызвать трудности.

В более сложных системах очевидным решением может показаться предоставление URI, позволяющих клиенту переходить между несколькими уровнями связей, например **`/customers/1/orders/99/products`**. Однако такой уровень сложности трудно обслуживать и адаптировать в случае дальнейшего изменения связей между ресурсами. Вместо этого постарайтесь сделать URI максимально простыми. Как только у приложения появляется ссылка на ресурс, оно может использовать эту ссылку для поиска элементов, связанных с указанным ресурсом. Предыдущий запрос можно заменить на URI **`/customers/1/orders`**, чтобы найти все заказы для клиента 1, а затем — на **`/orders/99/products`**, чтобы найти продукты в этом заказе.

Нередко можно видеть, как люди слепливают эти ресурсы вместе создавая 5-ти или 6-ти ярусные URL. Помните, что когда у вас есть первичный ключ для одного уровня, то, обычно, не нужно наслаивать уровни поверх, т.к. у вас уже есть конкретный объект. Другими словами, необходимость в URL более глубоких, чем приведенные выше, является исключением.

```
/ресурс/идентификатор/ресурс
```

Проектирование Web API | Sam’s Workbook (gasches.cc)

Конечные точки

Конечные точки являются важными аспектами взаимодействия с серверными веб-API, поскольку они определяют, где находятся ресурсы, к которым может получить доступ стороннее программное обеспечение. Обычно доступ осуществляется через URI, на который отправляются HTTP-запросы и от которого, таким образом, ожидается ответ. Веб-API могут быть общедоступными или частными, для последнего из которых требуется *токен доступа*.[6]

Конечные точки должны быть статичными, в противном случае корректное функционирование программного обеспечения, которое взаимодействует с ними, не может быть гарантировано. Если местоположение ресурса изменится (а вместе с ним и конечная точка), то ранее написанное программное обеспечение выйдет из строя, поскольку требуемый ресурс больше не может быть найден в том же месте. Поскольку поставщики API по-прежнему хотят обновлять свои веб-API, многие внедрили систему управления версиями в URI, указывающую на конечную точку.

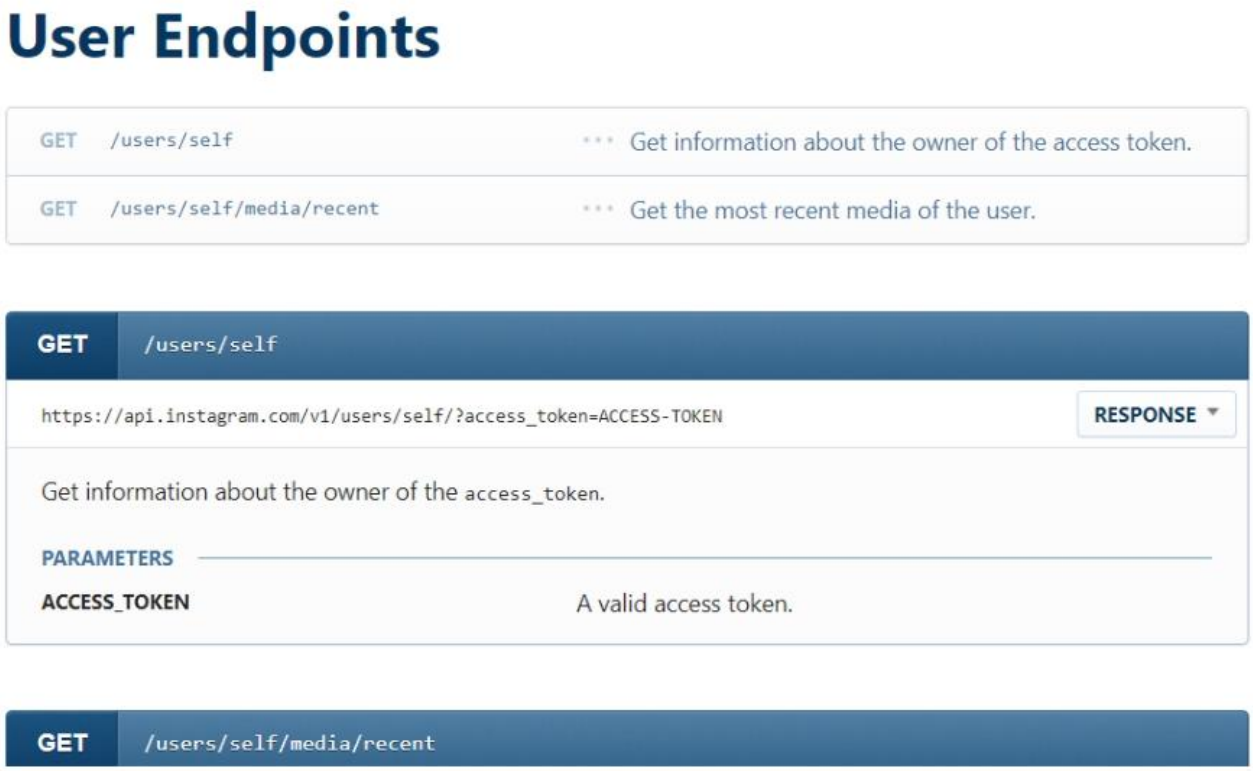
Ресурсы против сервисов [редактировать]

Веб 2.0 Веб-API часто используют машинные взаимодействия, такие как REST и SOAP. Веб-API RESTful используют методы HTTP для доступа к ресурсам с помощью параметров, закодированных в URL, и используют JSON или XML для передачи данных. В отличие от этого, протоколы SOAP стандартизированы W3C и требуют использования XML в качестве формата полезной нагрузки, обычно через HTTP. Кроме того, веб-API на основе SOAP используют проверку XML для обеспечения структурной целостности сообщений за счет использования XML-схем, предоставляемых документами WSDL. Документ WSDL точно определяет XML-сообщения и транспортные привязки веб-службы.

Для *API* конечная точка может включать *URL*-адрес сервера или службы. Каждая конечная точка — это место, из которого *API*-интерфейсы могут получить доступ к ресурсам, необходимым им для выполнения своих функций.

Один и тот же ресурс обычно имеет множество связанных конечных точек, каждая из которых имеет разные пути и методы, и возвращает различную информацию об одном и том же ресурсе. Конечные точки обычно имеют краткие описания, похожие на общее описание ресурса, только еще короче. Кроме того, конечная точка показывает только конечный путь URL ресурса, а не базовый, общий для всех конечных точек, путь.

Вот пример конечной точки ресурса `User` API Instagram



Как правило, API имеет несколько конечных точек, сгруппированных в одном ресурсе. В этом случае вы описываете как общий ресурс, так и отдельные конечные точки. Например, ресурс Campaigns имеет различные конечные точки, которые также описаны:

- POST `/campaigns`
- GET `/campaigns`
- GET `/campaigns/{campaign_id}`
- PATCH `/campaigns/{campaign_id}`
- DELETE `/campaigns/{campaign_id}`
- POST `/campaigns/{campaign_id}/actions/cancel-send`
- POST `/campaigns/{campaign_id}/actions/pause`
- POST `/campaigns/{campaign_id}/actions/replicate`
- POST `/campaigns/{campaign_id}/actions/resume`
- POST `/campaigns/{campaign_id}/actions/schedule`
- POST `/campaigns/{campaign_id}/actions/send`
- POST `/campaigns/{campaign_id}/actions/test`
- POST `/campaigns/{campaign_id}/actions/unschedule`

Tip: Когда разработчики создают API-интерфейсы, у них возникает вопрос проектирования: использовать много вариантов конечных точек (как в API-интерфейсе Eventbrite) или предоставить множество параметров для настройки одной и той же конечной точки. Луч-

ше всего найти баланс. Кажется, что тенденция заключается в предоставлении отдельных конечных точек, а не в предоставлении множества потенциально запутанных параметров для одной и той же конечной точки. С другой стороны, API-интерфейсы GraphQL предоставляют одну конечную точку различными способами запроса информации, содержащейся в конечной точке. См. Различия в GraphQL и REST, объясняемые на бургерах, для пояснения GraphQL и REST

Стоимость разработки простого API может составлять от $5 000 до $15 000, в то время как стоимость более сложного API может превысить $50 000 или даже $100 000, в зависимости от требований и задействованных ресурсов.

URI— (англ. Uniform Resource Identifier) унифицированный (единообразный) идентификатор ресурса. На английский манер произносится как [ю ар ай], по-русски чаще говорят [ури].

ПРИМЕР

https://openweathermap.org/api/one-call-3#history_daily_aggregation

One Call API 3.0 содержит 4 конечные точки и предоставляет доступ к различным данным:

- Текущая погода и прогнозы:
  - минутный прогноз на 1 час
  - Почасовой прогноз на 48 часов
  - Ежедневный прогноз на 8 дней

и правительственные предупреждения о погоде

- Данные о погоде для любой временной метки за 45 лет исторический архив и прогноз на 4 дня вперед
- Ежедневная агрегация данных о погоде за 45 лет архив и прогноз на 1,5 года вперед
- Обзор погоды с удобочитаемой сводкой погоды для прогноза на сегодня и завтрашний день, используя технологии искусственного интеллекта OpenWeather

1. Текущие и прогнозируемые данные о погоде

```
https://api.openweathermap.org/data/3.0/onecall?lat={lat}&lon={lon}&exclude={part}&appid={API key}
```

Параметры

`lat`	**обязательно**	Широта, десятичная дробь (-90; 90).
`lon`	**обязательно**	Долгота, десятичная дробь (-180; 180).
`appid`	**обязательно**	Ваш уникальный API-ключ (вы всегда можете найти его на странице своего аккаунта во вкладке «API-ключ»))
`exclude`	**необязательный**	С помощью этого параметра вы можете исключить некоторые части данных о погоде из API ответ. Это должен быть список, разделенный запятыми (без пробелов). Доступные значения: • `current` • `minutely` • `hourly` • `daily` • `alerts`
`units`	**необязательный**	Единицы измерения. , и единицы измерения доступны. Если параметр не используется, единицы измерения будут применяться по умолчанию.
`lang`	**необязательный**	Вы можете использовать этот параметр для получения выходных данных на вашем языке.

Пример вызова API

```
https://api.openweathermap.org/data/3.0/onecall?lat=33.44&lon=-94.04&exclude=hourly,daily&appid=c1b540f259064d8e68df0a0dd134ac43
```

2. Данные о погоде для любой временной метки

```
https://api.openweathermap.org/data/3.0/onecall/timemachine?lat={lat}&lon={lon}&dt={time}&appid={API key}
```

3. Ежедневная агрегация

```
https://api.openweathermap.org/data/3.0/onecall/day_summary?lat={lat}&lon={lon}&date={date}&appid={API key}
```

4. Обзор погоды

Вызов API

```
https://api.openweathermap.org/data/3.0/onecall/overview?lat={lat}&lon={lon}&appid={API key}
```

Пример вызова API

```
https://api.openweathermap.org/data/3.0/onecall/overview?lon=-11.8092&lat=51.509865&appid= c1b540f259064d8e68df0a0dd134ac43
```

Пример ответа API

```
{
   "lat": 51.509865,
   "lon": -0.118092,
   "tz": "+01:00",
   "date": "2024-05-13",
   "units": "metric",
   "weather_overview": "The current weather is overcast with a
temperature of 16°C and a feels-like temperature of 16°C.
The wind speed is 4 meter/sec with gusts up to 6 meter/sec
coming from the west-southwest direction.
The air pressure is at 1007 hPa with a humidity level of 79%.
The dew point is at 12°C and the visibility is 10000 meters.
The UV index is at 4, indicating moderate risk from the
sun's UV rays.
The sky is covered with overcast clouds, and there is
no precipitation expected at the moment.
Overall, it is a moderately cool and cloudy day
with light to moderate winds from the west-southwest."
}
```

Поля в ответе API

- `lat` Широта местоположения, десятичная дробь (−90; 90)
- `lon` Долгота местоположения, десятичная дробь (-180; 180)
- `tz` Часовой пояс в формате ±XX:XX
- `date` Дата, за которую формируется сводка в формате ГГГГ-ММ-ДД
- `units` Единицы измерения, указанные в запросе
- `weather_overview` Сгенерированный ИИ обзор погоды на запрашиваемую дату

Конечная точка примера будет выглядеть так:

GET /https://api.openweathermap.org/data/2.5/weather?zip=95050&units=imperial&appip=c1b540f259064d8e68df0a0dd134ac43

Параметры строки запроса отображаются после знака “?” и разделяются между собой амперсандом “&”. Порядок параметров в строке запросов значения не имеет.

Многие API передают ключ API в заголовке, а не в качестве параметра строки запроса в URL-адресе запроса.